

UNIVERSITY OF ENGINEERING AND TECHNOLOGY PESHAWAR

Department of Electrical Engineering

LABORATORY REPORT

Subject:	EE-170L Computer Programming Lab
Lab No:	Lab 08
Lab Title:	Two-Dimensional Arrays in C++
Semester:	Semester 2
Session:	Spring 2026

Submitted By:

Name: Muhammad Adil

Registration No: BF25NWELE0669

1. OBJECTIVE

To learn and apply two-dimensional arrays in C++ including declaration, initialization, accessing elements, calculating row and column sums, and finding maximum values in a matrix.

2. THEORY

A two-dimensional (2D) array is the simplest form of a multidimensional array. It can be visualized as a table with rows and columns. A 2D array is essentially a list of one-dimensional arrays. Syntax: type arrayName[rows][columns].

Key Concepts: (1) Declaration: `int matrix[3][3]` creates a 3x3 integer matrix. (2) Initialization: Can be done row by row using nested braces `{{1,2,3},{4,5,6},{7,8,9}}` or as a single list `{1,2,3,4,5,6,7,8,9}`. (3) Accessing: Elements are accessed using two subscripts: `array[row][column]`. (4) Nested loops are used to traverse 2D arrays - outer loop for rows, inner loop for columns.

3. LAB TASKS

3.1 Two-Dimensional Array Declaration, Initialization and Display

Declare a 2D integer array called 'matrix' with dimensions 3x3. Initialize the array with values 1, 2, 3, 4, 5, 6, 7, 8, and 9. Display the values of the array elements using nested loops.

Source Code:

```
#include <iostream>
using namespace std;

int main()
{
    // =====
    // 2D Array Declaration and Initialization
    // =====
    // Declare and initialize a 3x3 matrix with values 1 to 9
    // Nested braces indicate initialization for each row
    int matrix[3][3] = {
        {1, 2, 3},    // Row 0: First row elements
        {4, 5, 6},    // Row 1: Second row elements
        {7, 8, 9}     // Row 2: Third row elements
    };

    // =====
    // Display 2D Array Elements using Nested Loops
    // =====
    cout << "3x3 Matrix Elements:" << endl;
    cout << "===== " << endl;

    // Outer loop iterates through each row (i = 0 to 2)
    for (int i = 0; i < 3; i++)
```

```

    {
        // Inner loop iterates through each column (j = 0 to 2)
        for (int j = 0; j < 3; j++)
        {
            // Display element at position [i][j] with spacing
            cout << matrix[i][j] << "\t";

        }
        cout << endl; // New line after each row is complete
    }

    return 0; // Successful program termination
}

```

Expected Output:

```

3x3 Matrix Elements:
=====
1       2       3
4       5       6
7       8       9

```

3.2 Sum of Rows and Columns in a Two-Dimensional Array

Declare and initialize a 3x3 integer array with random values. Calculate and display the sum of each row and each column separately. Display the total sum of all the elements in the array.

Source Code:

```

#include <iostream>
using namespace std;

int main()
{
    // =====
    // 2D Array Declaration and Initialization
    // =====
    // Initialize a 3x3 array with sample values
    int matrix[3][3] = {
        {10, 20, 30}, // Row 0
        {40, 50, 60}, // Row 1
        {70, 80, 90}  // Row 2
    };

    int totalSum = 0; // Variable to store total sum of all elements
    int rowSum;      // Variable to store sum of current row
    int colSum;      // Variable to store sum of current column

    // =====
    // Display the Original Matrix

```

```

// =====
cout << "Original Matrix:" << endl;
for (int i = 0; i < 3; i++)
{
    for (int j = 0; j < 3; j++)
    {
        cout << matrix[i][j] << "\t";
    }
    cout << endl;
}
cout << endl;

// =====
// Calculate and Display Sum of Each Row
// =====
cout << "Sum of Each Row:" << endl;
for (int i = 0; i < 3; i++)
{
    rowSum = 0; // Reset row sum for each new row
    for (int j = 0; j < 3; j++)
    {
        rowSum += matrix[i][j]; // Add current element to row sum
    }
    cout << "Row " << i << " Sum = " << rowSum << endl;
    totalSum += rowSum; // Add row sum to total sum
}
cout << endl;

// =====
// Calculate and Display Sum of Each Column
// =====
cout << "Sum of Each Column:" << endl;
for (int j = 0; j < 3; j++)
{
    colSum = 0; // Reset column sum for each new column
    for (int i = 0; i < 3; i++)
    {
        colSum += matrix[i][j]; // Add current element to column sum
    }
    cout << "Column " << j << " Sum = " << colSum << endl;
}
cout << endl;

// =====
// Display Total Sum of All Elements
// =====
cout << "Total Sum of All Elements = " << totalSum << endl;

return 0;
}

```

Expected Output:

```
Original Matrix:
10      20      30
40      50      60
70      80      90
```

```
Sum of Each Row:
Row 0 Sum = 60
Row 1 Sum = 150
Row 2 Sum = 240
```

```
Sum of Each Column:
Column 0 Sum = 120
Column 1 Sum = 150
Column 2 Sum = 180
```

```
Total Sum of All Elements = 450
```

3.3 Finding the Maximum Value in a Two-Dimensional Array

Declare and initialize a 4x4 integer array with random values. Find and display the maximum value in the array along with its position.

Source Code:

```
#include <iostream>
using namespace std;

int main()
{
    // =====
    // 4x4 Array Declaration and Initialization
    // =====
    int matrix[4][4] = {
        {12, 45, 78, 23},    // Row 0
        {89, 56, 34, 91},    // Row 1
        {67, 99, 11, 44},    // Row 2
        {82, 15, 63, 77}     // Row 3
    };

    // Initialize max with first element of the array
    int max = matrix[0][0];
    int maxRow = 0, maxCol = 0; // Track position of maximum value

    // =====
    // Display the Matrix
    // =====
```

```

cout << "4x4 Matrix:" << endl;
cout << "======" << endl;
for (int i = 0; i < 4; i++)
{
    for (int j = 0; j < 4; j++)
    {
        cout << matrix[i][j] << "\t";
    }
    cout << endl;
}
cout << endl;

// =====
// Find Maximum Value in the Matrix
// =====
// Iterate through all elements to find maximum
for (int i = 0; i < 4; i++)
{
    for (int j = 0; j < 4; j++)
    {
        // Compare current element with max
        if (matrix[i][j] > max)
        {
            max = matrix[i][j]; // Update max if current is larger
            maxRow = i;         // Store row index of max
            maxCol = j;         // Store column index of max
        }
    }
}

// =====
// Display Result
// =====
cout << "Maximum Value in the Matrix: " << max << endl;
cout << "Position: Row " << maxRow << ", Column " << maxCol << endl;

return 0;
}

```

Expected Output:

```

4x4 Matrix:
=====
12      45      78      23
89      56      34      91
67      99      11      44
82      15      63      77

```

```

Maximum Value in the Matrix: 99

```

Position: Row 2, Column 1

4. CONCLUSION

This lab successfully demonstrated the concepts of two-dimensional arrays in C++. We learned how to declare, initialize, and access 2D array elements using nested loops. The tasks covered displaying matrix elements in tabular format, calculating row and column sums using separate nested loops, and finding the maximum value in a matrix by comparing each element. These operations are fundamental for matrix manipulation and form the basis for more complex data structures and algorithms in programming.